DATABASE RE-ENGINEERING · CASE STUDY

Applying the PACE Method

Re-engineering a production-planning stored procedure and migrating it from SQL Server to SAP HANA: a structural rebuild focused on robustness, maintainability and native performance.

SQL Server · T-SQL → SAP HANA · SQLScript

Author **Raúl García** Discipline **SQL / Data Engineering** Asset **1 stored procedure, 9 stages**

Method **Plan · Analyze · Construct · Evaluate**

~900 → ~700	14 → 6	9 → 1	Null-safe
Lines of code. The procedure was rebuilt, not ported.	Temporary tables consolidated into a clear pipeline.	Duplicated stage blocks unified into one set-based query.	Every division guarded; two latent logic bugs removed.

01 Introduction

The source system is a production indicator that answers one operational question: **how many units are needed at each production stage to meet the coming weeks of sales demand.** It works backwards through a nine-stage chain, from the lab to the nursery, weighting every stage by its **productivity (rate)** and its **quality**, under one governing identity: **real factor = quality × rate**. Moving it to SAP HANA could not be done one-to-one, because T-SQL and SQLScript differ in syntax, functions and typing. The migration was therefore treated as a structured re-engineering, guided end to end by the **PACE** method.

02 The PACE walkthrough

Four phases took the procedure from a fragile monolith to a native, configurable HANA solution.

P

PHASE 01 · PLAN / PREPARE

Frame the problem and lock the business rules

Inventory the legacy asset and pin down exactly which logic has to be preserved before touching code.

- **Legacy asset:** a single ~900-line T-SQL procedure with 14 temporary tables, deeply nested scalar subqueries and nine near-identical `UNION ALL` blocks.
- **Rules to preserve 1:1:** the per-stage safety-stock percentages, the full stage chain and the identity `factor = quality × rate`.
- **Target state:** reproduce the results faithfully while making the procedure modular, null-safe and driven by SAP master data instead of hardcoded constants.

A

PHASE 02 · ANALYZE

Map every incompatibility and risk

Separate what actually breaks on HANA from what is merely fragile, and decide what to keep versus redesign.

DIALECT GAPS (NOT IN HANA)

- `IIF()`, `DATEADD / GETDATE / DATEPART`
- Inline `DECLARE @x = (SELECT…)` assignments
- Session `#temp` tables and `COLLATE` clauses
- Implicit type coercion between text and numbers

STRUCTURAL RISKS

- Raw divisions across the stage chain, so one zero or NULL broke the run
- Clone-fallback logic copy-pasted nine times
- Stage windows hardcoded per product as fixed day ranges
- Magic numbers scattered through the calculations

> **Found during analysis:** the copy-paste had already drifted. In one stage block both `WHEN` branches tested the same condition, so the second could never run; in two others, inventory was checked against a hardcoded clone instead of the input parameter. Both bugs were removed in the rebuild rather than migrated.

PHASE 03 · CONSTRUCT

C Rebuild in SQLScript: modular, dynamic, null-safe

Rewrite the logic natively and remove the anti-patterns rather than carrying them across.

- **Native SQLScript:** `CASE` for IIF, `ADD_DAYS / CURRENT_DATE / WEEK` for dates, `SELECT … INTO` for variables, explicit `TO_DECIMAL / TO_INT / TO_NVARCHAR` casting for HANA's strict typing.
- **Deduplicated:** the nine fallback blocks collapse into one `COALESCE("FinalFactor","DefaultFactor")` inside a single factors table.
- **Consolidated pipeline:** 14 temp tables reduced to 6 clear steps: defects → activities → quality → rates → factors → output.
- **Data-driven durations:** hardcoded windows moved to SAP configuration tables (`@STAGE_DURATION` , `@STAGE`), with each window computed as `weeks × 7` .
- **Parameterized by clone:** the whole run is driven by one input via `LIKE :v_clone || '%'` .

● BEFORE · T-SQL · UNGUARDED

```
-- raw division, NULL-prone
DECLARE @S09 FLOAT = (
 SELECT ((@WeeklyDemand * Weeks)
          / RealFactor)
 FROM #FinalFactors
 WHERE StageCode = 9)
```

● AFTER · SQLSCRIPT · NULL-SAFE

```
SELECT IFNULL(
  (v_12WEEKS * "StageWeeks")
  / NULLIF(COALESCE(
      "FinalFactor","DefaultFactor"), 0),
0)
INTO v_S09 FROM #TEMP_FACTORS
WHERE "StageCode" = '09';
```

PHASE 04 · EVALUATE

E Validate parity and measure the gains

Confirm the numbers still match the original, then quantify the structural improvement.

- **Output parity:** the same staged report, with minimum and differentiation units per stage plus programmed vs. projected production by week, month and year.
- **Simpler:** ~900 → ~700 lines, 14 → 6 temp tables, 9 → 1 fallback expression.
- **More robust:** no divide-by-zero or NULL failures, safe by construction, and two latent logic bugs removed.
- **More maintainable:** stage durations and factors are now configuration, not code; analysts adjust SAP tables instead of editing the procedure.

03 Migration decisions at a glance

The concrete translation of each concern from T-SQL to native SQLScript.

Concern	SQL Server (before)	SAP HANA (after)
Conditional	`IIF(cond, a, b)`	`CASE WHEN cond THEN a ELSE b END`
Dates	`DATEADD(WW,-52,GETDATE())`	`ADD_DAYS(CURRENT_DATE, -364)`
Variable load	`DECLARE @x = (SELECT …)`	`SELECT … INTO v_x FROM …`
Temp storage	`CREATE TABLE #t (14 of them)`	`LOCAL TEMPORARY TABLE (6)`
Stage fallback	`9 × UNION ALL blocks`	`COALESCE(clone, default)`
Durations	`hardcoded day ranges per stage`	`@STAGE_DURATION → weeks × 7`
Division safety	`a / b`	`a / NULLIF(b,0) + IFNULL / COALESCE`
Parameterization	`IIF branch per clone`	`LIKE :v_clone || '%'`
Typing	`implicit coercion`	`TO_DECIMAL / TO_INT / TO_NVARCHAR`

Conclusion and final implementation

PACE turned a risky, impossible-to-port monolith into a native SAP HANA procedure that is materially simpler, safe against runtime failures and configurable by the business: the same results on a far stronger foundation.

The delivered procedure runs on SAP HANA with per-clone parameterization, stage durations sourced from SAP configuration tables and null-safe calculations end to end, producing consistent weekly production requirements across every stage.

All excerpts are sanitized: table names, custom fields, item-code conventions and business constants are replaced with generic placeholders. Seven side-by-side transformations are detailed in the companion Technical Deep-Dive.